



Software Requirements Specification
Version 1.0

SonicSentinel AI

Project Name: SonicSentinel AI

Theme: AcousticX Intelligence

Category: NextWave AI and ML

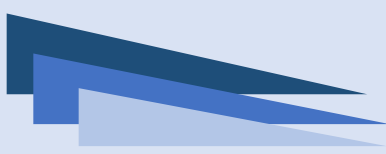


Table of Contents

1.1 Background and Necessity for the Application.....	2
1.2 Proposed Solution.....	3
1.3 Purpose of the Document	17
1.4 Scope of Project.....	17
1.5 Constraints.....	18
1.6 Functional Requirements	18
1.7 Non-Functional Requirements.....	30
1.8 Competition Integrity and Anti-Shortcut Requirements	31
1.9 Interface Requirements	32
1.9.1 Hardware	32
1.9.2 Software.....	33
1.10 Project Deliverables	34

1.1 Background and Necessity for the Application

Sound-event monitoring is important in manufacturing plants, public spaces, transportation facilities, commercial buildings, homes, farms, service centers, and security-sensitive environments. Critical sounds such as machinery faults, gunshots, glass breaking, alarms, panic screams, aggression, and calls for help may indicate equipment failure, accidents, vandalism, violence, emergencies, or threats requiring immediate attention.

Traditional sound monitoring depends on people continuously listening to live audio or manually reviewing recordings after an event has occurred. This process is time-consuming and prone to missed sounds, delayed responses, inconsistent decisions, and false alarms, particularly in noisy or crowded environments. **SonicSentinel AI** aims to automate and improve this process through Python-based Web development, audio processing, machine learning, Google Teachable Machine, and intelligent event-analysis techniques.

The ‘**SonicSentinel AI**’ reduces the requirement for completely manual sound monitoring by automatically analyzing uploaded audio clips and live microphone input. The application detects sound events such as Machinery Fault, Glass Breaking, Alarm or Siren, Vehicle Horn, Animal Sound, Gunshot, Panic Scream, Aggression, Person Asking for Help, or Background Noise.

The application generates confidence scores using a Python classification model and compares the result with a separately trained Google Teachable Machine audio model. It also evaluates audio quality, detects uncertain or overlapping sounds, assigns event severity, and generates alerts for critical events. The goal is to deliver an accurate, secure, responsive, and accessible solution that supports faster sound-event detection and informed decision-making.





1.2 Proposed Solution

The proposed solution introduces an AI-powered, Web-based, and real-time sound-event detection application that helps users, security teams, maintenance personnel, and administrators identify important sounds accurately and efficiently.

Users can either upload an audio clip or enable live microphone monitoring through the Web application. The application validates the audio, extracts file information, removes or reduces unwanted noise, divides the audio into fixed-duration segments, and extracts meaningful acoustic features.

The Python classification model analyzes the pre-processed audio and predicts the detected sound category. The same audio segment is independently analyzed using a separately trained Google Teachable Machine audio model. Both models generate confidence scores for all supported classes.

The application compares the predicted categories and confidence scores produced by the two models. It also generates a waveform and spectrogram, evaluates recording quality, detects overlapping or uncertain sounds, applies event-specific rules, and generates alerts for critical events.

The application must support the following mandatory sound categories:

1. Machinery Fault
2. Glass Breaking
3. Alarm or Siren
4. Vehicle Horn
5. Animal Sound
6. Gunshot
7. Panic Scream
8. Aggression or Violent Conflict
9. Person Asking for Help
10. Background Noise

Teams may add optional categories such as:

- Drone Sound
- Drilling or Grinder Sound
- Fireworks
- Vehicle Backfire
- Explosion
- Door Impact
- Graffiti Spray

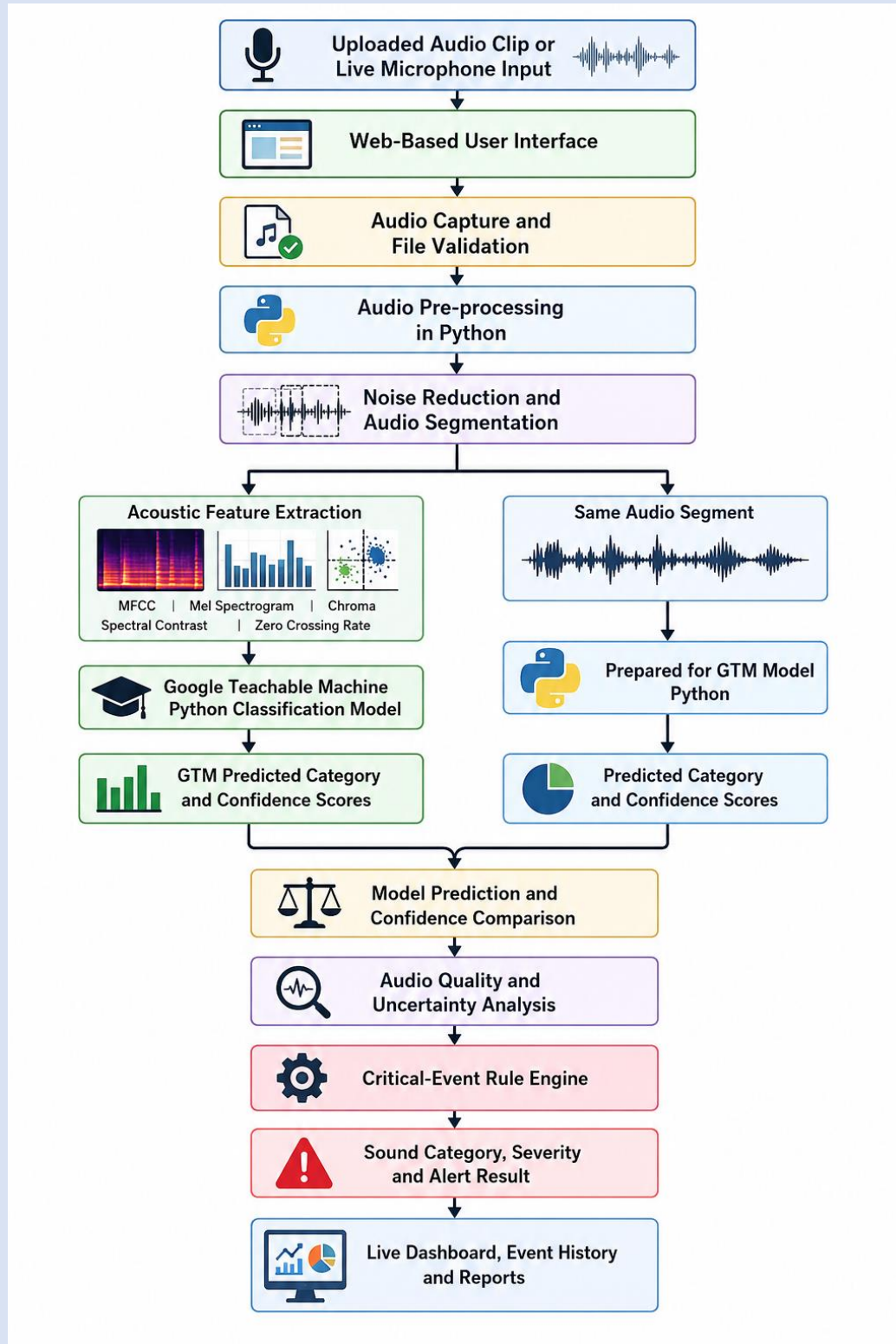


- Normal Machinery
- Unknown Sound

The application must not treat itself as a certified emergency-response or law-enforcement system. It is a competition prototype intended for controlled and ethical testing.



The sample architecture for **SonicSentinel AI** application is as follows:



Sample Architecture of the Application



Reference Application

Students may refer to:

<https://soundeventdetector.eu/>

to understand the general concept of real-time sound-event recognition, critical-event monitoring, environmental-noise analysis, security alerts, event history, and user-facing detection workflows.

The reference solution detects and alerts sounds such as gunshots, glass breaking, panic screams, aggression, drills, grinders, drones, alarms, and other abnormal events. It also demonstrates real-time information delivery and event-monitoring workflows.

The reference application must not be copied or reproduced. Teams must design and develop their own original Python-based Web application by implementing all the functional requirements specified in this document.

The application must include:

- Uploaded audio analysis
- Live microphone monitoring
- Python sound classification
- Google Teachable Machine classification
- Confidence-score comparison
- Waveform and spectrogram generation
- Audio-quality assessment
- Critical-event detection
- Real-time alerts
- Manual review
- Event history
- Dashboard and reports

Development Phase of the Application

The development phase of the application includes the following steps:

Step 1: Audio Data Collection

Teams must collect, create, or ethically source labelled audio recordings for all ten mandatory sound categories.



Each recording must be assigned a unique Audio ID and should include metadata such as:

- Filename
- Sound category
- Duration
- Sampling rate
- Number of channels
- Recording environment
- Recording device
- Approximate source distance, where known
- Original or augmented status
- Dataset split

Step 2: Uploaded Audio and Live Microphone Capture

The application must support two input modes:

- Uploaded audio-file analysis
- Live microphone monitoring

Uploaded clips may be analyzed as complete recordings or divided into smaller segments. Live microphone audio must be captured continuously in short windows suitable for real-time prediction.

Step 3: Audio File Validation

The application must validate:

- Audio format
- File size
- Duration
- Sampling rate
- Channel count
- File integrity
- Presence of an audio signal
- Microphone availability
- Permission status

Silent, damaged, unsupported, excessively short, or unusable recordings must be rejected with appropriate messages.

Step 4: Audio Pre-Processing

The uploaded or captured audio must be prepared using Python.



Pre-processing should include:

- Resampling
- Stereo-to-mono conversion
- Amplitude normalization
- Silence trimming
- Noise reduction
- Fixed-duration segmentation
- Padding or truncation
- Audio-format conversion, where required

Step 5: Common Audio Dataset Creation

One common underlying sound dataset must be created for both classification models.

For the Python model, the original recordings and extracted acoustic features must be used.

For Google Teachable Machine, the same underlying recordings must be divided into short audio samples and placed under the corresponding sound classes.

Every original recording and its derived segments must retain the same Audio ID and class label.

Step 6: Acoustic Feature Extraction

The Python pipeline must extract meaningful acoustic features such as:

- Mel-frequency cepstral coefficients
- Mel spectrogram
- Chroma features
- Zero-crossing rate
- Root mean square energy
- Spectral centroid
- Spectral bandwidth
- Spectral roll-off
- Onset strength
- Tempo, where relevant

Step 7: Python Model Training

Teams must train and compare at least three suitable machine learning or deep learning models.



Possible models include:

- Support Vector Machine
- Random Forest
- Gradient Boosting
- XGBoost
- Convolutional Neural Network
- Convolutional Recurrent Neural Network
- Transfer-learning audio model

The best-performing model must be selected using:

- Accuracy
- Precision
- Recall
- F1-score
- Macro F1-score
- Confusion matrix
- Class-wise performance
- Critical-event recall
- Noise-robustness results

Step 8: Python Sound Classification

The Python model must classify each uploaded or live audio segment and generate confidence scores for all ten mandatory classes.

Step 9: Google Teachable Machine Model Training

Teams must create a Google Teachable Machine (GTM) Audio Project.

Google Teachable Machine can classify short sound samples recorded through a microphone or provided through files.

The GTM model must use the same class names as the Python model:

- Machinery Fault
- Glass Breaking
- Alarm or Siren
- Vehicle Horn
- Animal Sound
- Gunshot
- Panic Scream
- Aggression



- Person Asking for Help
- Background Noise

The GTM training samples must originate from the same underlying training recordings used by the Python model.

Step 10: Google Teachable Machine Classification

The same uploaded recording or live audio window must be prepared for the trained GTM model.

The GTM model must independently generate:

- Predicted sound category
- Confidence score for every category

The Python prediction must not be provided to the GTM model.

Step 11: Model Prediction and Confidence Comparison

The application must compare:

- Python predicted category
- GTM predicted category
- Python confidence for all classes
- GTM confidence for all classes
- Predicted-class match status
- Top-class confidence difference
- Top-two confidence difference

The confidence difference must be calculated as:

```
Confidence Difference =  
|Python Top-Class Confidence  
- Google Teachable Machine Top-Class Confidence|
```

Since the two models are independently trained, exact confidence equality is not required.

Step 12: Real-Time Sound Monitoring

The application must continuously capture live microphone audio in fixed-duration windows, such as one to three seconds.



Each live audio window must be:

- Captured
- Validated
- Preprocessed
- Analyzed by the Python model
- Analyzed by the GTM model
- Compared
- Checked against alert rules
- Displayed on the live dashboard

Step 13: Audio Quality Analysis

The application must check each recording for:

- Silence
- Clipping
- Excessive noise
- Low signal strength
- Unsuitable duration
- Encoding problems
- Missing audio frames

The audio quality should be classified as:

- Good
- Acceptable
- Poor
- Unusable

Step 14: Critical-Event Recognition

The application must identify critical acoustic events such as:

- Gunshot
- Glass breaking
- Panic scream
- Aggressive shouting
- Violent conflict
- Person asking for help
- Alarm or siren
- Serious machinery fault



The application should attempt to distinguish similar events such as:

- Gunshot vs. fireworks
- Gunshot vs. vehicle backfire
- Panic scream vs. normal shouting
- Aggression vs. normal conversation
- Glass breaking vs. metal impact
- Alarm vs. vehicle horn
- Machinery fault vs. normal machinery
- Help request vs. ordinary speech

Step 15: Repeated Detection Confirmation

Critical alerts should not depend only on one unstable prediction.

The application should confirm a critical event using one or more of the following:

- Detection in consecutive audio windows
- Minimum confidence threshold
- Agreement between the Python and GTM models
- Repeated detection within a configured time period
- Minimum difference between the top two classes
- Acceptable audio quality

Step 16: Alert Generation

The application must generate alerts for critical events when the configured conditions are satisfied.

Alert severity should be one of the following:

- Informational
- Low
- Medium
- High
- Critical

Examples:

- Background Noise: Informational
- Vehicle Horn: Low or Medium
- Animal Sound: Low
- Machinery Fault: High



- Glass Breaking: High
- Alarm or Siren: High
- Aggression: High
- Panic Scream: Critical
- Person Asking for Help: Critical
- Gunshot: Critical

Step 17: Manual Review

The following recordings must be routed for manual review:

Different predictions from the two models

- Low confidence
- Poor audio quality
- Similar top-class confidence scores
- Overlapping sounds
- Unsupported sound pattern
- Critical event without sufficient model agreement
- Possible false alarm

Step 18: Dashboard and Event History

The application must display:

- Microphone monitoring status
- Current detected sound
- Python prediction and confidence
- GTM prediction and confidence
- Model agreement status
- Waveform
- Spectrogram
- Audio quality
- Event severity
- Alert status
- Recent detections
- Critical-event timeline
- Manual-review events
- Category statistics



Step 19: Final Sound Event Decision

The final result must be generated by considering:

- Python model prediction
- GTM model prediction
- Confidence-score difference
- Top-two confidence difference
- Audio quality
- Repeated detection
- Critical-event rules
- Manual-review conditions

The final result must include:

- Detected sound category
- Model agreement status
- Confidence level
- Event severity
- Alert status
- Recommended action
- Manual-review requirement

Step 20: Event Storage and Reporting

The application must store:

- Audio ID
- Audio metadata
- Python prediction
- Python confidence scores
- GTM prediction
- GTM confidence scores
- Audio-quality result
- Severity
- Alert status
- Reviewer decision
- Model versions
- Processing timestamp



Hint

For this project, teams must create or collect their own common sound-event dataset containing a minimum of **3,000 unique original audio clips**. No complete competition-ready dataset will be provided.

The dataset must contain approximately equal recordings for the following classes:

- 300 Machinery Fault clips
- 300 Glass Breaking clips
- 300 Alarm or Siren clips
- 300 Vehicle Horn clips
- 300 Animal Sound clips
- 300 Gunshot clips
- 300 Panic Scream clips
- 300 Aggression clips
- 300 Person Asking for Help clips
- 300 Background Noise clips

The dataset must include variations in:

- Recording device
- Recording distance
- Indoor and outdoor environment
- Sound intensity
- Background interference
- Echo and reverberation
- Audio duration
- Clean and noisy conditions
- Single and overlapping events
- Near and distant recordings

Different speakers, where speech is involved

The Person Asking for Help class must be limited to defined safety phrases such as:

- "Help me"
- "Somebody help"
- "Please help"
- "Call for help"
- "Emergency"



These samples must be recorded voluntarily, generated synthetically where permitted, or obtained from ethically licensed sources.

The common dataset must be divided using a stratified split:

- **70% Training:** 2,100 original audio clips
- **15% Validation:** 450 original audio clips
- **15% Testing:** 450 original audio clips

The same underlying split must be used for both models:

- The Python model must use only training recordings and derived training features.
- GTM must use only audio samples derived from the training recordings.
- Validation and testing samples must not be used for either model's training.

The same unseen test recordings must be used to compare the two models.

Audio clips may be divided into shorter segments for training. All segments derived from one original recording must remain in the same dataset split.

Training audio may be augmented using:

- Background-noise addition
- Time shifting
- Pitch shifting
- Time stretching
- Volume adjustment
- Limited reverberation
- Distance simulation
- Recording-device simulation

Augmented recordings must remain in the same split as the original recording. Augmented recordings must not be counted as unique original clips.



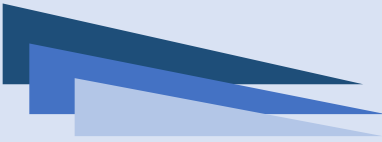
1.3 Purpose of the Document

This document outlines the development plan for the **SonicSentinel AI** application using audio processing, Artificial Intelligence, Machine Learning, and Google Teachable Machine. It ensures that stakeholders and developers have a common understanding of the project requirements, promotes effective communication, and provides a structured roadmap for developing a reliable sound-event detection application.

Key areas include identifying critical sounds in real time, reducing manual audio monitoring, generating confidence-based predictions, and comparing Python and Google Teachable Machine results. The application also provides timely alerts for events such as gunshots, glass breaking, panic screams, aggression, machinery faults, and calls for help. This document is intended for both stakeholders and developers.

1.4 Scope of Project

The scope of the project includes developing a complete Machine Learning pipeline for the **SonicSentinel AI** application. The project aims to automate sound-event detection by analyzing uploaded audio clips and live microphone input using audio-processing and machine-learning techniques. It extracts acoustic features, generates waveform and spectrogram visualizations, and classifies sounds using Python and Google Teachable Machine models.



The application compares confidence scores and identifies events such as machinery faults, glass breaking, alarms, vehicle horns, animal sounds, gunshots, panic screams, aggression, calls for help, and background noise. The application also generates severity-based alerts, routes uncertain results for manual review, and presents live detections, event history, and reports through a user-friendly Web interface.



1.5 Constraints

The project may face several constraints, such as variations in audio quality, background noise, recording distance, microphone type, and overlapping sound events. Application accuracy depends heavily on the size, balance, diversity, and reliability of the training dataset, as well as the available computational resources. Differences may also occur between the predictions and confidence scores generated by the Python model and Google Teachable Machine. Ensuring audio privacy, obtaining user consent for microphone access, and avoiding bias across different speakers, environments, and recording devices are also critical limitations to consider.

1.6 Functional Requirements

Creating a **SonicSentinel AI** application involves several steps and components. The following are the functional requirements:

- i. **User Registration and Authentication** - The application should allow users, analysts, reviewers, security personnel, maintenance personnel, and administrators to register and securely log in.



ii. Role-Based Access Control – The application should provide separate access permissions for:

- Normal user
- Audio reviewer
- Security operator
- Maintenance operator
- Administrator

iii. User Profile Management – Users should be able to create and update their profile information. The application should assign a unique User ID to every registered user.



iv. Audio File Upload – Users should be able to upload recordings in formats such as:

- WAV
- MP3
- FLAC
- OGG
- M4A

v. Batch Audio Upload – Authorized users should be able to upload multiple audio clips for batch classification.

vi. Live Microphone Monitoring – The application should allow users to start and stop live microphone monitoring after granting permission.



vii. Microphone Status Display – The application should display whether the microphone is:

- Available
- Active
- Paused
- Disconnected
- Permission denied

viii. Audio File Validation – The application should validate:

- Format
- Size
- Duration
- Sampling rate



- Channel count
- File integrity
- Presence of usable sound

ix. Audio Preview – Users should be able to play, pause, replay, seek, and control the volume of uploaded recordings.



x. Audio Metadata Extraction – The application should extract:

- Filename
- Format
- Duration
- Sampling rate
- Number of channels
- Bit depth, where available
- File size
- Upload date and time

xi. Audio Pre-processing – The application should perform:

- Resampling
- Mono conversion
- Amplitude normalization
- Silence trimming
- Noise reduction
- Segmentation
- Padding
- Truncation



xii. Silence Detection – The application should detect silent or near-silent recordings and prevent misleading classification.

xiii. Clipping Detection – The application should detect severe clipping and display an audio-quality warning.

xiv. Background Noise Estimation – The application should estimate the amount of background interference in the recording.

xv. Audio Segmentation – Long recordings and live input should be divided into fixed-duration segments.



The application should store the start and end timestamps of each segment.

xvi. Common Sound Dataset – The project should use a common dataset containing all ten mandatory sound classes.

xvii. Dataset Metadata Management – The dataset should maintain:

- Audio ID
- Filename
- Class label
- Source
- Duration
- Recording environment
- Original or augmented status
- Dataset split



xviii. Dataset Balance Checking – The dataset-preparation pipeline should identify class imbalance and insufficient sample counts.

xix. Audio Augmentation – The training pipeline should support controlled augmentation such as:

- Noise addition
- Time shifting
- Pitch shifting
- Time stretching
- Volume variation
- Reverberation



xx. Acoustic Feature Extraction – The Python pipeline should extract:

- MFCC
- Mel spectrogram
- Chroma
- Zero-crossing rate
- Root mean square energy
- Spectral centroid
- Spectral bandwidth
- Spectral roll-off



- xxi. Waveform Generation** – The application should generate and display the waveform of uploaded and selected live-recorded segments.
- xxii. Spectrogram Generation** – The application should generate and display a spectrogram or Mel spectrogram.
- xxiii. Python Model Training** – Teams should train and compare at least three suitable classification models.
- xxiv. Hyperparameter Tuning** – The training process should include systematic tuning of relevant model parameters.
- xxv. Python Sound Classification** – The selected Python model should predict one of the ten mandatory sound classes.
- xxvi. Python Confidence Scores** – The Python model should generate confidence scores for all supported classes.
- xxvii. Google Teachable Machine (GTM) Audio Model Training** – The application should use a separately trained Google Teachable Machine audio model
- xxviii. GTM Model Integration** – The trained GTM model should be exported or linked and integrated into the Web application using a supported method.
- xxix. GTM Sound Classification** – The GTM model should independently classify the same audio segment processed by the Python model.
- xxx. GTM Confidence Scores** – The application should display GTM confidence scores for all supported sound classes.
- xxxi. Prediction Comparison** – The application should compare the Python and GTM predicted categories.
- xxxii. Confidence Comparison** – The application should calculate the absolute difference between the models' top-class confidence scores.





xxxiii. Model Consistency Status – The application should display one of the following:

- Acceptable Match
- Weak Match
- Model Disagreement
- Uncertain Result



xxxiv. Top-N Prediction Display – The application should display at least the top three predictions from each model.

xxxv. Configurable Confidence Threshold – Administrators should be able to configure the minimum confidence required for automatic event



xxxvi. Top-Two Margin Threshold – The application should allow administrators to configure the minimum difference required between the highest and second-highest class confidence.

xxxvii. Audio Quality Classification – The application should classify audio quality as:

- Good
- Acceptable
- Poor
- Unusable

xxxviii. Uncertain Sound Detection – A result should be marked uncertain when:

- Confidence is below threshold
- Top classes have similar confidence
- Audio quality is poor
- The models disagree
- Overlapping sounds are detected



xxxix. Overlapping Sound Detection – The application should identify possible overlapping events when several classes receive significant confidence.

xl. Repeated Event Confirmation – Critical events should be confirmed across consecutive windows or repeated detections where applicable.



- xli. Machinery Fault Detection** – Machinery Fault should generate a maintenance-related alert and recommended inspection action.
- xlii. Glass Breaking Detection** – Glass Breaking should generate a high-severity security alert when confidence requirements are satisfied.
- xliii. Alarm or Siren Detection** – Alarm or Siren should generate a high-severity attention alert.
- xliv. Vehicle-Horn Detection** – Vehicle Horn should be stored as a traffic or environmental sound event.
- xlv. Animal-Sound Detection** – Animal Sound should be recorded as a non-critical or context-dependent event.
-  **xlvi. Gunshot Detection** – Gunshot should generate a critical alert only after configured confirmation and confidence rules are satisfied.
- xlvii. Panic-Scream Detection** – Panic Scream should generate a critical safety alert when detected with sufficient confidence.
- xlvi. Aggression Detection** – Aggression or Violent Conflict should generate a high or critical alert depending on confidence and repeated detection.
- xlvi. Person Asking for Help Detection** – The application should detect defined help phrases represented in the training dataset and generate a critical
 - 1. Background Noise Handling** – Background Noise should normally be treated as non-critical unless the estimated noise level exceeds a configured limit.
 - li. Unknown Sound Handling** – Sounds that cannot be classified reliably should be marked as Unknown or Manual Review Required.



lii. Alert Severity Assignment – The application should assign:

- Informational
- Low
- Medium
- Critical



liii. Configurable Alert Rules – Administrators should be able to configure:

- Critical categories
- Minimum confidence
- Repeated-detection requirement
- Audio-quality requirement
- Model-agreement requirement
- Alert severity
- Recommended action

liv. Real-Time Alert Generation – Critical events should generate visible alerts on the dashboard.



lv. Alert Acknowledgement – Authorized users should be able to acknowledge, dismiss, or escalate alerts.

lvi. Alert History – The application should maintain a complete history of alerts and user actions.

lvii. Manual Review Queue – Uncertain and conflicting events should be routed to a manual-review queue.

lviii. Reviewer Audio Playback – Reviewers should be able to listen to the relevant audio segment.

lix. Reviewer Decision – Reviewers should be able to confirm or correct the detected class.

lx. Reviewer Comments – Authorized reviewers should be able to add comments and recommended actions.

lxi. Decision Override – A reviewer should be able to override the automatic result while preserving the original model outputs.





lxii. Event Status Tracking – The application should support statuses such as:

- Uploaded
- Classified
- Uncertain
- Alert Generated
- Manual Review
- Reviewed
- Closed

lxiii. User Dashboard – The user dashboard should display:

- Recent uploads
- Current detections
- Critical events
- Audio-quality warnings
- Manual-review records



lxiv. Live Monitoring Dashboard – The live dashboard should display:

- Microphone status
- Current sound
- Python result
- GTM result
- Model agreement
- Event severity
- Active alert

lxv. Administrator Dashboard – The administrator dashboard should display:

- Total audio events
- Events by category
- Critical alerts
- Average confidence
- Model disagreements
- Poor-quality recordings
- Detection trends

lxvi. Event Timeline – The application should display detected sound events on a chronological timeline.



lxvii. Search and Filtering – Users should be able to filter by:

- Audio ID
- Filename
- Sound category
- Date range
- Confidence range
- Severity
- Quality
- Review status
- User



lxviii. Analytics and Reporting – The application should provide analytics on:

- Category frequency
- Critical-event frequency
- Confidence distribution
- False positives
- False negatives
- Model disagreement
- Audio quality
- Alert response

lxix. Downloadable Analysis Report – The report should include:

- Audio metadata
- Python prediction
- GTM prediction
- Confidence scores
- Confidence difference
- Waveform
- Spectrogram
- Audio quality
- Severity
- Alert status
- Review decision

lxx. Data Export – Administrators should be able to export records in CSV or Excel-compatible format.



lxxi. Secure Audio Storage – Uploaded audio and captured event segments should be stored securely with controlled access.

lxxii. Database Storage – The database should store:

- Users
- Audio metadata
- Predictions
- Confidence scores
- Alerts
- Reviews
- Model versions
- Audit records

lxxiii. Duplicate Audio Detection – The application should calculate a secure hash and identify identical audio uploads.

lxxiv. Near-Duplicate Detection – The application should attempt to identify re-encoded, trimmed, or volume-adjusted copies of existing audio.

lxxv. Model Version Tracking – Each prediction should be linked to the Python and GTM model versions used.

lxxvi. Audit Trail – The application should record:

- Logins
- Uploads
- Microphone sessions
- Predictions
- Alerts
- Reviews
- Overrides
- Exports
- Model updates



lxxvii. Error Handling – The application should display understandable errors for:

- Invalid files
- Unsupported formats



- Audio-decoding failure
- Model failure
- Database failure
- Report failure



lxxviii. Monitoring and Anomaly Alerts – Administrators should be alerted about:

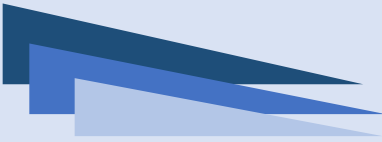
- Repeated failed uploads
- Model failures
- Unusual low-confidence spikes
- Excessive critical alerts
- Duplicate files
- Failed login attempts

lxxix. Privacy Controls – Users should be informed when the microphone is active. The application should not secretly record audio.

lxxx. Data Retention – Administrators should be able to configure how long audio files and event records are retained.

In addition, the application should provide a responsive and user-friendly Web interface accessible through desktop, tablet, and supported mobile browsers.





1.7 Non-Functional Requirements

There are several non-functional requirements that should be fulfilled by the application.

1. **Performance:** The application should process and classify an uploaded audio clip of up to 30 seconds within eight seconds under normal operating conditions. Live sound windows should produce a prediction within three seconds.
2. **Scalable:** The application should store and manage at least 20,000 sound-event records and support multiple concurrent users without significant performance degradation.
3. **Usable:** The application should provide an intuitive and user-friendly Web interface for normal users, reviewers, security operators, maintenance personnel, and administrators.
4. **Accuracy:** Both models should target an overall test accuracy of at least 85% and a macro F1-score of at least 0.80. Critical classes such as Gunshot, Glass Breaking, Panic Scream, Aggression, and Person Asking for Help should target a recall of at least 85%.
5. **Available:** The application should remain operational during required monitoring and evaluation hours with at least 99% uptime.



These are the bare minimum expectations from the project. It is a must to implement the FUNCTIONAL and NON-FUNCTIONAL requirements given in this SRS.

Once they are complete, you can use your own creativity and imagination to add more features if required.



1.8 Competition Integrity and Anti-Shortcut Requirements

1. Each team member must explain the modules assigned to them.
2. The GitHub repository must contain meaningful commits across all five competition days.
3. Teams must maintain a development log showing:
 - Work completed
 - Problems encountered
 - Dataset changes
 - Model failures
 - Code changes
 - Tests performed
4. Evaluators may select any function, class, preprocessing step, feature, or model component and ask the team to explain it.
5. Teams must complete a surprise modification such as:
 - Adding a new sound category
 - Changing the confidence threshold
 - Supporting a new audio format
 - Modifying segment duration
 - Adding an alert rule
 - Changing the repeated-detection requirement
 - Adding a dashboard filter
6. Teams may be required to fix a deliberately introduced defect.
7. Hard-coded predictions, invented confidence values, unexplained code, hidden decision APIs, or hard-coded test answers may result in disqualification.
8. The application will be tested using unseen recordings.



9. Hidden audio tests may contain:
 - Background noise
 - Echo
 - Low volume
 - Different devices
 - Partial events
 - Overlapping sounds
 - Similar sound categories
 - Re-encoded files
 - Distant sound sources
10. AI tools used during development must be declared in AI_USAGE.md.
11. AI-generated output cannot replace technical understanding.
12. The final sound classification must not be generated through an external generative-AI API.
13. Teams must demonstrate that the Python model and GTM model independently generate their predictions.
14. The Python model's prediction or confidence must not be supplied as an input to the GTM model.

1.9 Interface Requirements

1.9.1 *Hardware*

Intel Core i5/i7 Processor or higher
8/16 GB RAM or higher
Color SVGA monitor
500 GB Hard Disk space
Mouse and Keyboard

1.9.2 *Software*

Technologies to be used:

1. **Frontend:** HTML5, CSS3, JavaScript, Bootstrap, Streamlit, React, or another suitable frontend technology
2. **Backend:** Flask, Django, or FastAPI
3. **Audio Files and Data Store:** WAV, MP3, FLAC, OGG, M4A, CSV, JSON, spectrogram images, and extracted-feature files
4. **Database:** Firebase, MongoDB, MySQL, PostgreSQL, SQLite, or another relational or NoSQL database
5. **Programming/IDE:** Python, Jupyter Notebook, Anaconda, Google Colab, PyCharm, or Visual Studio Code
6. **Machine Learning:** Scikit-learn, XGBoost, TensorFlow, Keras, PyTorch, and Google Teachable Machine
7. **Audio Libraries:** Librosa, SoundFile, Pydub, SciPy, TorchAudio, Audioread, or other suitable audio libraries
8. **Libraries:** NumPy, Pandas, Matplotlib, Plotly, Joblib, Pickle, and Pillow
9. **Testing:** pytest, unittest, Postman, and browser developer tools
10. **Version Control:** Git and GitHub
11. **Deployment:** Render, Railway, PythonAnywhere, Streamlit Community Cloud, or another suitable hosting platform





1.10 Project Deliverables

Each team must design, build, test, document, deploy, and demonstrate the complete **SonicSentinel AI** application.

1. Project Report

The report must include:

- Problem definition
- Background and business necessity
- Proposed solution
- Purpose
- Scope
- Assumptions
- Constraints
- Functional requirements
- Non-functional requirements
- Application architecture
- Module descriptions
- Database design
- Data dictionary
- Data Flow Diagram
- Use Case Diagram
- Activity Diagram
- Sequence Diagram
- Decision Flow Diagram
- Audio-processing pipeline
- Critical-event rule design
- Python model design
- GTM model design
- Dataset description
- Dataset-collection method
- Data augmentation
- Audio preprocessing
- Feature extraction
- Python model training
- GTM model training
- Model evaluation



- Confusion matrices
- Accuracy, precision, recall, and F1-score
- Model prediction comparison
- Confidence comparison
- Noise-robustness analysis
- False-positive analysis
- False-negative analysis
- Testing strategy
- Security
- Privacy
- Limitations
- Future enhancements

2. Source Code

The public GitHub repository should include:

README.md

AI_USAGE.md

requirements.txt

LICENSE

src/

templates/

static/

data/

audio_dataset/

notebooks/

python_models/

gtm_model/

feature_extraction/

audio_preprocessing/

augmentation/

alert_rules/

database/



tests/
sample_audio/
documentation/
screenshots/
reports/
config/

The source submission must include:

- Python frontend and backend files
- HTML, CSS, JavaScript, or Streamlit files
- Database scripts
- Audio preprocessing scripts
- Feature-extraction scripts
- Augmentation scripts
- Python model-training scripts
- Saved Python model
- GTM exported model or integration files
- Label files
- Waveform code
- Spectrogram code
- Alert-rule files
- Test scripts
- Sample audio

3. Dataset

The dataset submission must include:

- Training audio
- Validation audio
- Testing audio
- Dataset metadata
- Audio IDs
- Class labels
- Source information
- Original or augmented status
- Dataset split
- Dataset statistics



- Data dictionary
- Augmentation scripts
- Quality report

4. Python Classification Model Evidence

Teams must submit:

- Training dataset
- Preprocessing scripts
- Feature-extraction scripts
- Feature descriptions
- Models tested
- Hyperparameters
- Training results
- Validation results
- Testing results
- Confusion matrix
- Accuracy
- Precision
- Recall
- F1-score
- Macro F1-score
- Class-wise performance
- Critical-class recall
- Saved model
- Preprocessing pipeline
- Model version
- Sample predictions

At least three models must be compared.

5. Google Teachable Machine Evidence

Teams must submit:

- GTM project link or permitted evidence
- Screenshots of all classes
- Training sample counts
- Background Noise samples
- Training configuration
- Training observations



- Incorrect classifications
- Retraining details
- Exported GTM model or model link
- Metadata and label files
- Model version
- Test screenshots
- Integration evidence

6. Model Prediction and Confidence Comparison Report

The comparison report must contain at least **100 unseen test recordings**, with at least ten from every class.

The report must include:

- Audio ID
- Filename
- Actual class
- Python predicted class
- Python confidence for every class
- GTM predicted class
- GTM confidence for every class
- Class-match status
- Top-class confidence difference
- Top-two confidence margin
- Audio quality
- Severity
- Alert status
- Manual-review status
- Final decision
- Correct or incorrect result
- Explanation of major disagreement
- Overall comparison summary

7. Alert Rule Files

Configurable rules must include:

- Sound category
- Minimum confidence
- Top-two margin
- Required consecutive detections



- Model-agreement requirement
- Audio-quality requirement
- Severity
- Recommended action
- Manual-review condition
- Escalation condition

Rules must be stored in JSON, YAML, CSV, or a database.

8. Test Cases

Tests must include:

- Functional tests
- Integration tests
- Boundary tests
- Negative tests
- Security tests
- Database tests
- Audio-format tests
- Microphone tests
- Silence tests
- Clipping tests
- Noise tests
- Preprocessing tests
- Feature-extraction tests
- Python model tests
- GTM model tests
- Comparison tests
- Alert-rule tests
- Duplicate-audio tests
- Low-confidence tests
- Overlapping-sound tests
- Hidden-test readiness checklist

The project must demonstrate:

- Every mandatory sound class
- Silent audio
- Invalid audio
- Low-quality audio



- Low-confidence result
- Model disagreement
- Overlapping sound
- Duplicate audio
- High-severity alert
- Manual-review case
- Real-time detection

9. Installation Instructions

The documentation must provide:

- Prerequisite software
- Supported operating system
- Python version
- Virtual-environment steps
- Dependency installation
- FFmpeg installation, where required
- Audio-library configuration
- Database configuration
- Database initialization
- Python model placement
- GTM model placement or URL configuration
- Environment variables
- Folder setup
- Execution command
- Test command
- Administrator setup
- Troubleshooting

10. Execution Instructions

The **README.md** must explain:

- Installation
- Registration and login
- Audio upload
- Audio preview
- Live microphone monitoring
- Metadata viewing
- Waveform viewing



- Spectrogram viewing
- Python prediction
- GTM prediction
- Confidence interpretation
- Model comparison
- Audio-quality interpretation
- Alert acknowledgement
- Manual review
- Dashboard access
- Event-history search
- Report export
- Automated-test execution

11. GitHub Repository

The repository must:

- Be publicly accessible
- Contain meaningful commits
- Include commits from all members
- Include complete code
- Include model instructions
- Include dataset instructions
- Include permitted sample audio
- Avoid unlicensed or confidential recordings
- Include assumptions
- Include limitations
- Include evaluator credentials
- Include screenshots
- Include test results
- Include report, blog, and video links

12. Deployed Application

Teams should submit:

- Public application URL
- Evaluator credentials
- Administrator credentials
- Sample audio
- Testing instructions



Where deployment is not possible, complete local execution instructions must be provided.

13. Demonstration Video

The mandatory **.mp4** video must demonstrate:

- Login
- Audio upload
- Live microphone monitoring
- File validation
- Metadata
- Preprocessing
- Waveform
- Spectrogram
- Python prediction
- Python confidence
- GTM prediction
- GTM confidence
- Model comparison
- Confidence difference
- Audio quality
- Repeated detection
- Critical alert
- Alert acknowledgement
- Manual review
- Reviewer override
- Dashboard
- Event history
- Report generation
- Every sound class
- Low-confidence case
- Model-disagreement case
- Noisy case
- Overlapping-sound case
- Real-time critical detection



14. Technical Blog

The blog must contain at least 2,000 words and discuss:

- Business problem
- Background
- Proposed solution
- Architecture
- Dataset collection
- Ethical sourcing
- Dataset challenges
- Audio preprocessing
- Feature extraction
- Python model development
- GTM training
- Models compared
- Confidence comparison
- Waveforms
- Spectrograms
- Alert rules
- Real-time monitoring
- Difficulties
- False positives
- False negatives
- Model disagreement
- Noise robustness
- Security
- Privacy
- Limitations
- Lessons learned
- Future enhancements

15. AI Tool Usage Declaration

Any AI tool used must be declared in **AI_USAGE.md**.

The declaration must include:

- Tool name
- Purpose
- Assistance requested



- Files affected
- Student modifications
- Testing completed
- Verifying team members

Teams must not submit AI-generated code without understanding, reviewing, modifying, and testing it.

The final sound classification must be generated by the Python and GTM models, not by a generative-AI API.

16. Final Submission Checklist

The final submission must include:

- Project report
- Public GitHub URL
- Complete source code
- Training dataset
- Validation dataset
- Testing dataset
- Dataset metadata
- Python model
- GTM model
- Preprocessing scripts
- Feature-extraction scripts
- Alert rules
- Model-comparison report
- Installation instructions
- Execution instructions
- Deployment URL
- Demonstration video
- Technical blog
- `AI_USAGE.md`
- Team contribution record

Documentation is a very important part of the project. All aspects of the application, dataset, preprocessing, models, comparison, alerts, testing, installation, execution, security, privacy, and limitations must be documented completely and clearly.

~~~ End of Document ~~~